

Deep Learning Optimisé - Jean Zay

Optimizing large models – part 1

INSTITUT DU DÉVELOPPEMENT ET DES RESSOURCES EN INFORMATIQUE SCIENTIFIQUE

CC BY NC SA

- Aujourd'hui on va parler de l'optimisation des gros modèles en machine learning
- Dans cette Partie 1 : d'où viennent les gros modèles, et comment les faire tourner en inférence.
- Fil rouge : où sont tous ces paramètres → pourquoi ça ne tient plus sur 1 GPU, qu'est-ce qu'on peut y faire

→ *Transition* : on commence par les gros modèles eux-mêmes.

The Very Large Models

Transformers ◂

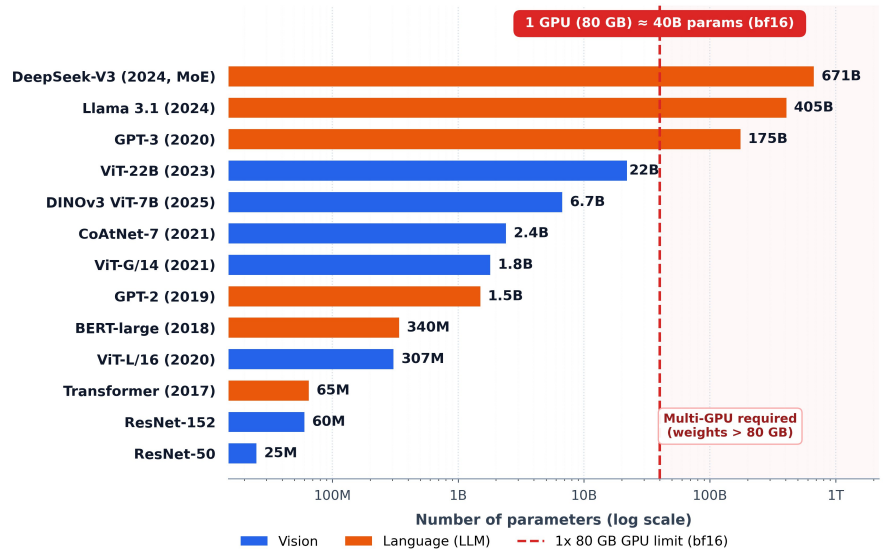
Vision Transformers (ViT) ◂

CoAtNet ◂

- Pas un cours de NLP : intuition + ordres de grandeur.
- Transformer = brique de presque tous les gros modèles (texte, son, et image -> on parle de vision transformer)
- CoAtNet = hybride CNN + Transformer

→ *Transition* : d'abord, ils sont gros comment ?

Larger and larger models

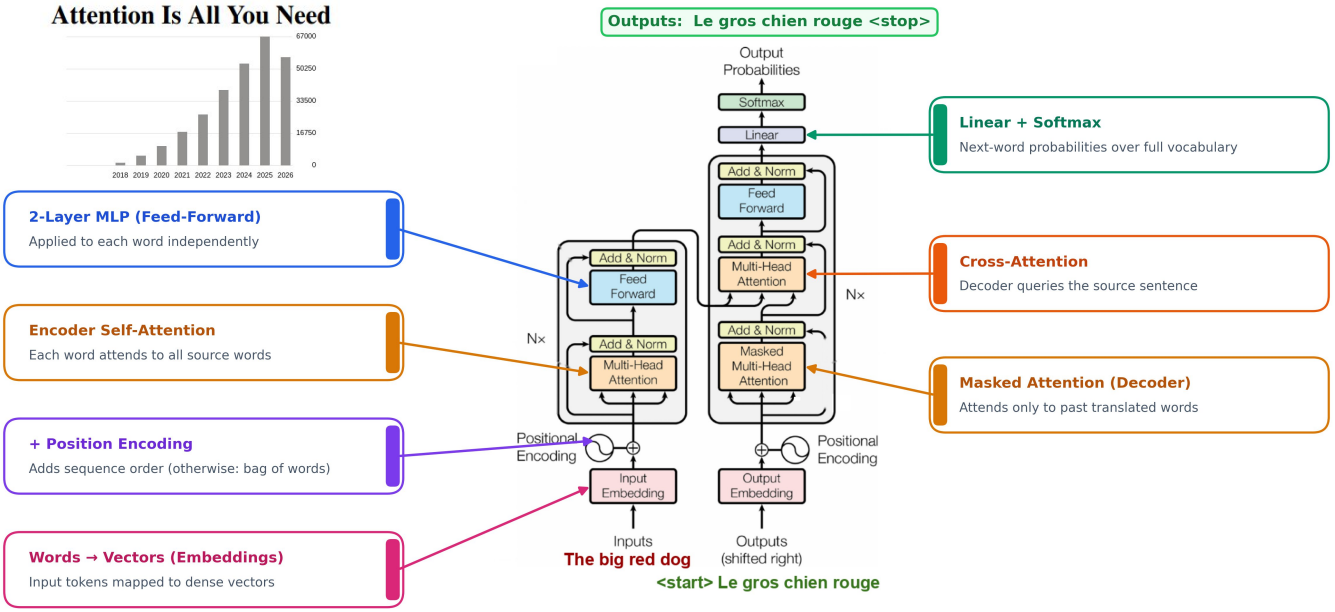


– Vision: from 60 M (ResNet-152) to several billions
– Language: hundreds of billions of parameters
– Reference: 1 GPU 80 Go ≈ 40 B parameters in BF16 ... weights only!

- Repère connu : ResNet-50 = 25 M, ResNet-152 = 60 M.
- Vision : ViT-G 1,8 B (2021), DINOv3 6,7 B (2025), jusqu'à ViT-22B.
- Langage : 1 à 2 ordres de grandeur au-dessus.
- Calcul de tête : bf16 = 2 octets/param → 80 Go ≈ 40 B params. Et ce ne sont que les poids : sans gradients, optimiseur ni activations.
- CoAtNet-7 (famille du modèle du TP) : n° 1 ImageNet en 2021.

→ *Transition* : tous ces gros modèles reposent sur la même brique, née en 2017… pour la traduction.

The first Transformer (Vaswani et al.,2017)

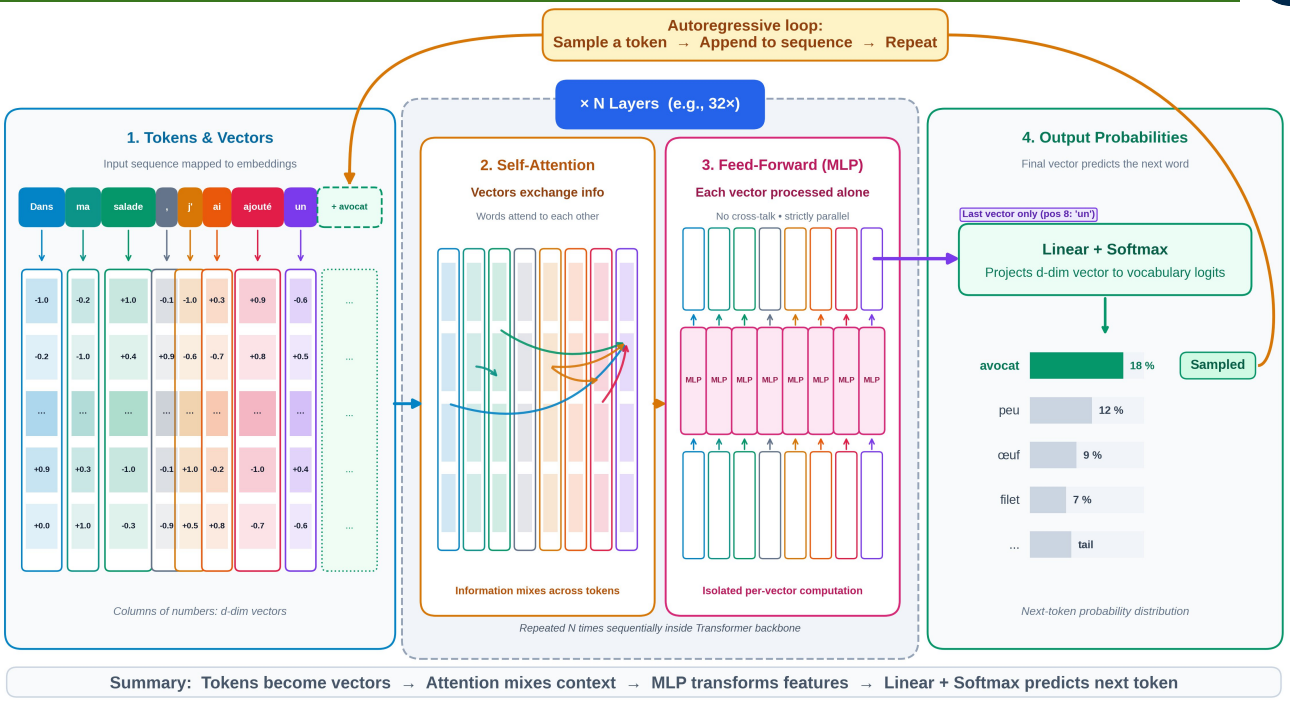


No more recurrence → Entire sequence processed in parallel → Ideal for GPUs

- Google, 2017. Tâche : traduction (anglais → allemand / français).
- Avant : les RNN lisaient mot par mot. C'était séquentiel donc lent sur GPU, et toute la phrase était résumée dans 1 vecteur (goulot). Ce papier : popularise l'architecture transformer, qui repose sur le mécanisme d'attention,
- L'attention existait déjà en complément des RNN (2014). Ici : l'attention seule.
- Gros avantage : toute la phrase est traitée d'un coup → parallèle → GPU.
- Parcours :
* mots → vecteurs + position ;
* l'encodeur comprend la phrase source ;
* le décodeur écrit mot par mot, en regardant ce qu'il a déjà écrit et la phrase source.
- Sortie : une probabilité pour chaque mot du vocabulaire.
- Taille : ~65 M params (version base), soit à peu près un ResNet-152.

→ *Transition* : les LLM actuels n'en gardent qu'une moitié, le décodeur. Voici comment ça marche, en une slide.

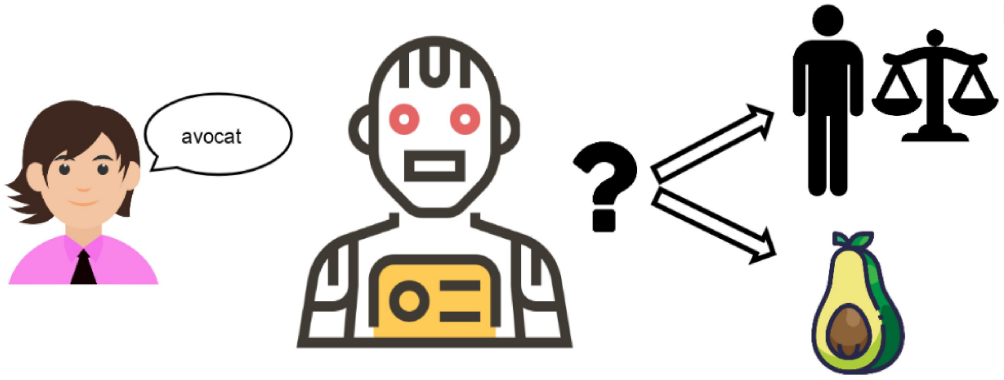
Transformer in one slide



- Texte → tokens (morceaux de mots). Chaque token → 1 vecteur, lu dans une table apprise.
- Les directions de l'espace portent du sens (reine – roi ≈ femme – homme).
- Par la magie des espaces à high dim...
- Puis N fois (Llama 3 70B : N = 80) :
* Attention : les vecteurs échangent de l'info → le contexte entre dans chaque vecteur ;
* MLP : chaque vecteur est traité seul → « réfléchir », appliquer des connaissances.
- À la fin : le dernier vecteur → un score par token du vocabulaire → softmax → probabilités.
- Génération : tirer un token, l'ajouter, relancer → token par token
- Entraînement : prédire le token suivant, sur des milliers de milliards de tokens (~15 000 Llama 3).
- Tout ça, ce sont des multiplications de matrices → GPU.

→ *Transition* : la pièce clé, c'est l'attention. Pourquoi on en a besoin : un exemple.

Attention: intuition (1/2)



Ambiguity: which meaning without context?

The Problem with Static Embeddings

1. Fixed token representation

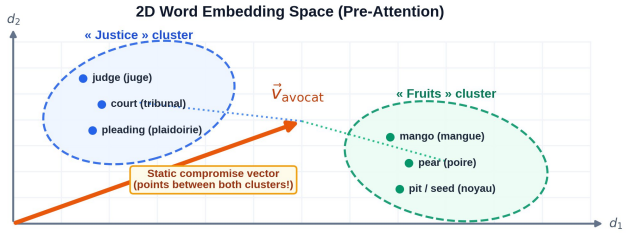
Before attention, each word has exactly one static vector.

2. Lexical polysemy

In French, 'avocat' means both lawyer and avocado.

3. Absence of sentence context

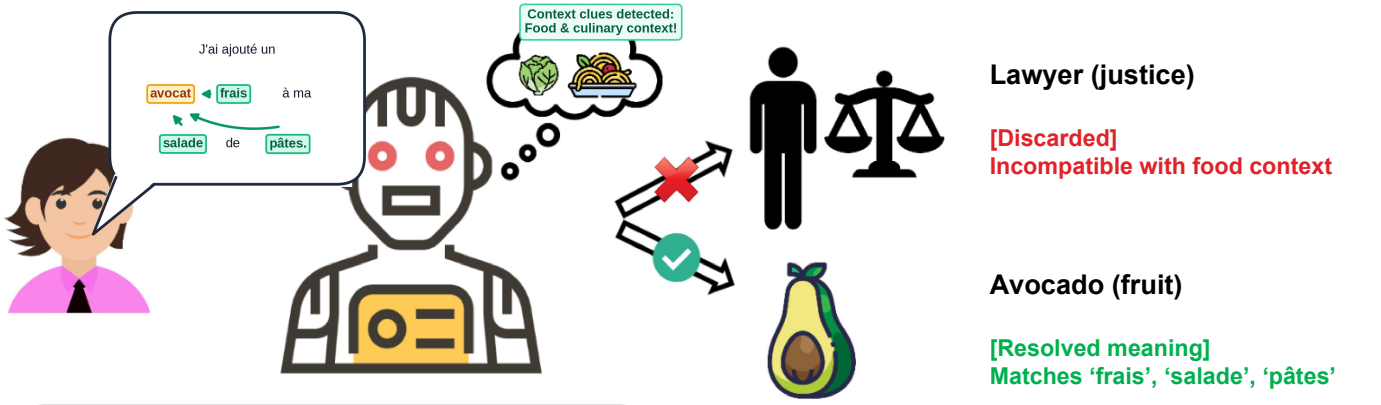
The model doesn't know if we are in court or in the kitchen!



- Rappel : 1 token = 1 vecteur lu dans une table → le même dans toutes les phrases.
- Homographe « avocat » : fruit ou métier → le vecteur mélange les deux sens.
- Il faut donc que le contexte puisse modifier le vecteur.

→ *Transition* : c'est justement le rôle de l'attention.

Attention: intuition (2/2)



How Self-Attention Solves Polysemy

1. Contextual attention score

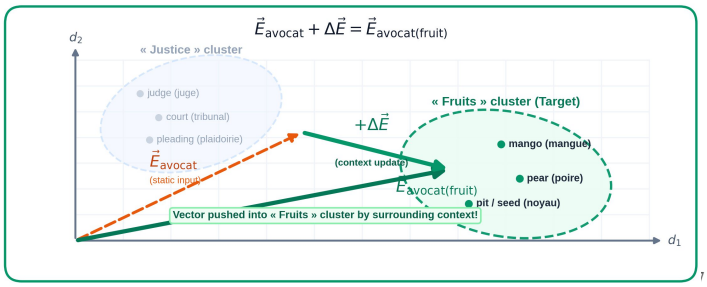
The word 'avocat' attends heavily to 'frais', 'salade', 'pâtes'.

2. Context update vector (ΔE)

A correction vector ΔE is computed by blending context values.

3. Disambiguated representation

Adding ΔE shifts 'avocat' directly into the culinary fruit cluster!

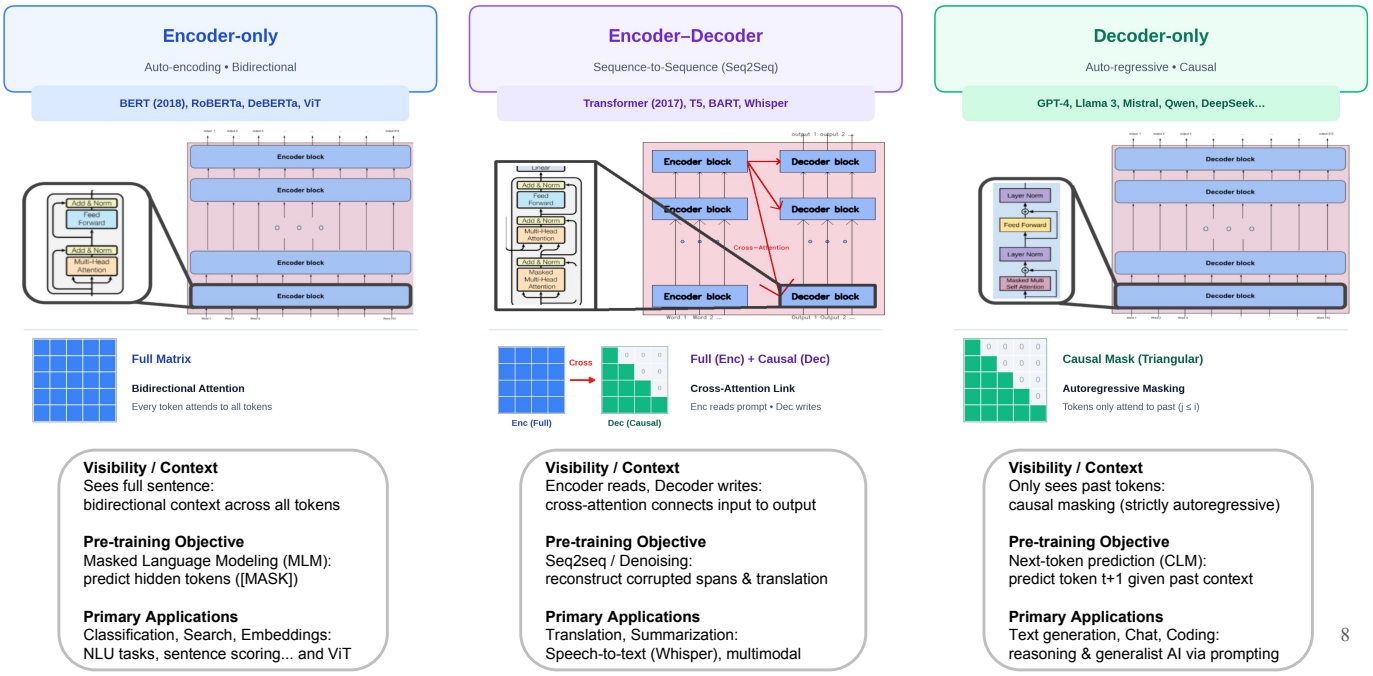


- « salade », « pâtes », « frais » → sens fruit ; « tribunal », « plaidoirie » → sens métier.
- Chaque mot repère les mots utiles pour lui et leur « emprunte » de l'info.
- Le vecteur n'est pas remplacé : on lui *ajoute* une correction (connexion résiduelle).
- Pas que les homographes :

* « gros » + « chien » → le vecteur de « chien » encode alors « gros chien » ;
* contexte lointain aussi (sujet du texte, qui parle…).

→ *Transition* : qui a le droit de regarder qui ? Toute la phrase, ou seulement le passé → 2 grandes familles.

Three families of Transformers



- 2017 : encodeur + décodeur (traduction). Ensuite, chaque famille garde une moitié.
- BERT (Google, 2018) :
* encodeur seul : chaque mot voit toute la phrase ;
* très bon pour comprendre et classer (la phrase entière ou chaque token) ;
* a lancé la « transformer-mania ». ViT est de la même famille.
- GPT (OpenAI, 2018) :
* décodeur seul + masque causal : chaque token ne voit que les précédents → génération ;
* bonus du masque : à l'entraînement, toutes les positions prédisent leur token suivant *en parallèle*.
- L'encodeur-décodeur est toujours là : T5, Whisper.
- Constat avec GPT-2/GPT-3 (2019–2020) : à grande échelle, prédire le token suivant suffit pour tout faire via le prompt (classer, traduire, coder) → tous les LLM sont des décodeurs.

→ *Transition* : encodeur ou décodeur, le calcul d'attention est le même. Le voici.

Self-Attention: computation

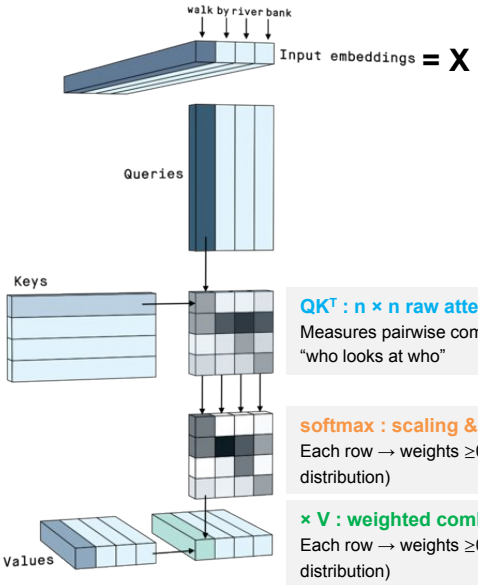
$Q = \mathbf{X}W_Q$, $K = \mathbf{X}W_K$, $V = \mathbf{X}W_V$

$\text{Attention}(Q, K, V) = \text{softmax}(QK^T/\sqrt{d_k})\, V$

QUERY (Q)
"What am I looking for?"
e.g. for "bank" (or "avocado"): "a clue about my meaning?"

KEY (K)
"What do I contain?"
e.g. for "river" (or "salad"): "nature / waterway!"

VALUE (V)
"What do I transmit if selected?"
the contextual representation passed to the enrich the attended token



QK^T : n × n raw attention scores
Measures pairwise compatibility, "who looks at who"

softmax : scaling & normalization
Each row → weights ≥0, sum=1 (attention distribution)

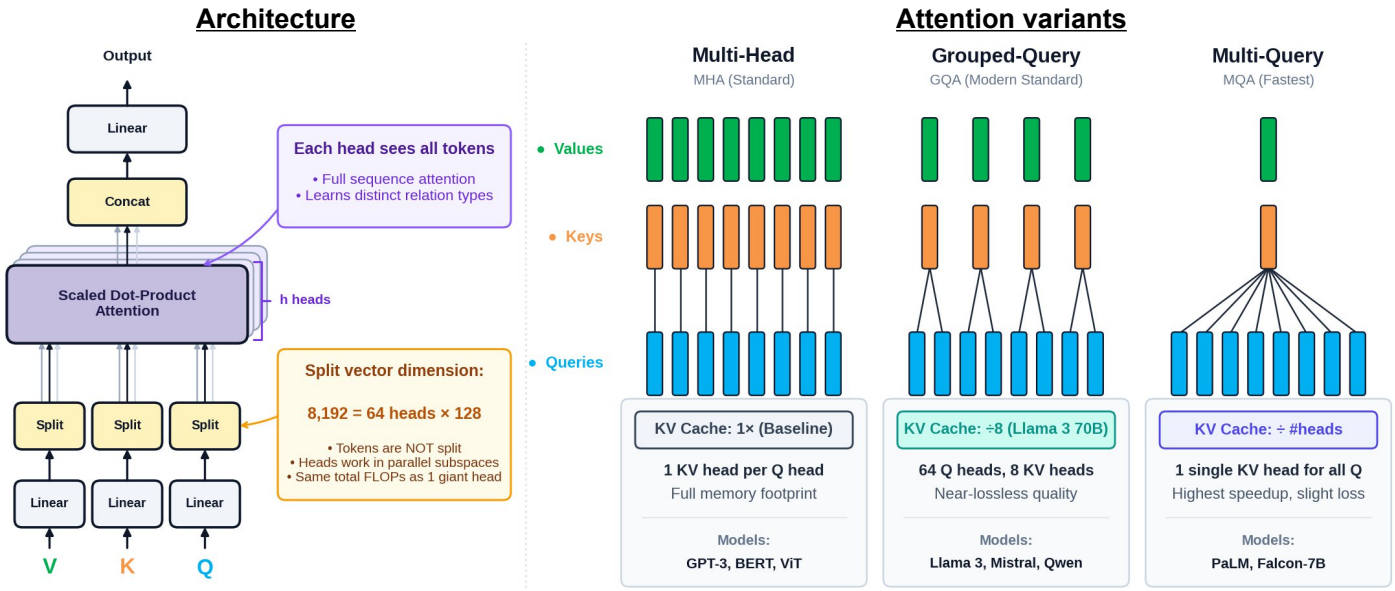
× V : weighted combination of values
Each row → weights ≥0, sum=1 (attention distribution)

+ DECODER Causal attention masking
– Future attention score set to $-\infty$
– Tokens only attend to current and past positions

Quadratic complexity:
n × n attention matrix → $O(n^2)$ cost

- Entrée X : les n vecteurs de la séquence.
- mult by 3 matrices apprises → 3 versions de chaque vecteur : Q (question), K (étiquette), V (contenu).
* Même entrée, mais Q ≠ K ≠ V
- Score = produit scalaire Q·K : grand si la question de l'un correspond à l'étiquette de l'autre.
- Dans la figure, « bank » = berge ou banque → même problème qu'« avocat ». « river » répond à sa question.
- Toutes les paires de mots d'un coup : QKT, une matrice n × n.
- ÷ $\sqrt{d_k}$: sinon les scores sont trop grands → softmax trop « pointu » → apprentissage instable.
- Softmax par ligne → des poids.
- × V → chaque token reçoit un mélange des Values, ajouté à son vecteur.
- Décodeur : masque causal = un triangle (futur à $-\infty$).
- HPC :
* que des matmuls → idéal pour le GPU ;
* mais matrice n x n → mémoire et calcul en n^2 → les contextes longs coûtent cher (→ FlashAttention & co)
→ *Transition* : une attention = un type de relation. On en fait tourner plusieurs en parallèle.

Multi-Head Attention… and GQA

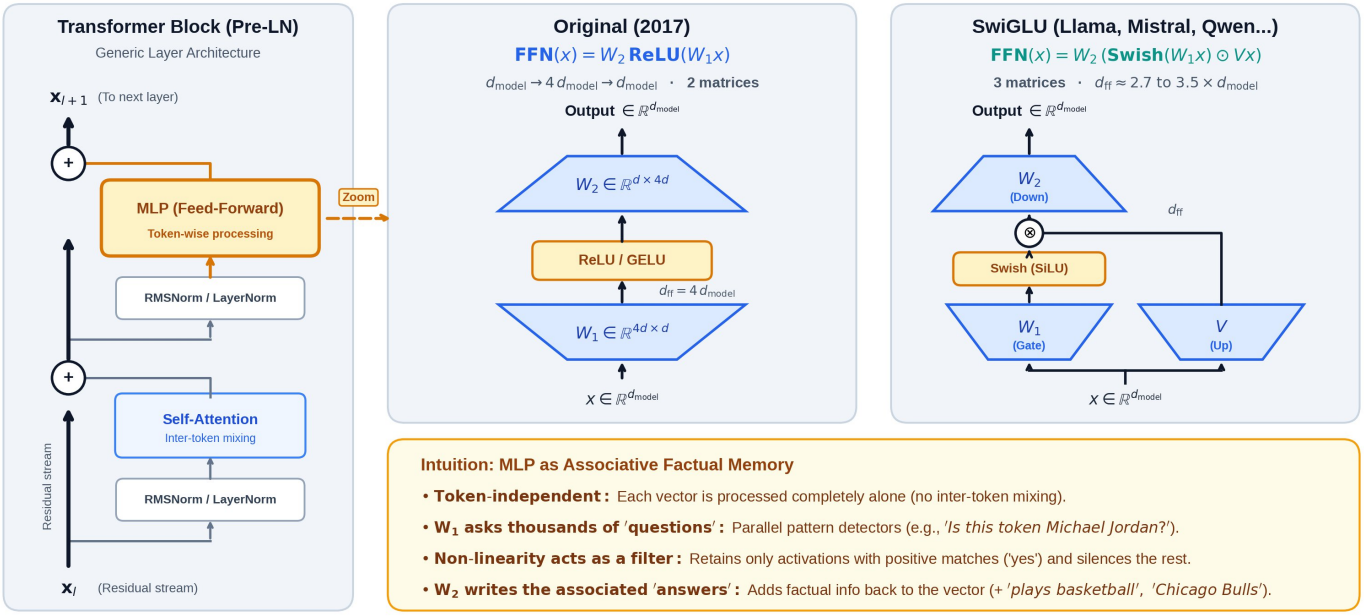


Fewer K, V heads → Smaller KV cache in generation (cf. KV cache bottleneck)

- 1 tête (W matrix) ≈ 1 type de relation (adjectif → nom, pronom → personne, sens d'un mot…)
- Plusieurs têtes en parallèle : 96 par couche pour GPT-3, 64 pour Llama 3 70B.
- Sur quoi bosse les têtes ? Pas toute la séquence, on découpe
- Mais on ne coupe pas la séquence, on coupe la *dimension* d'encodage :
* chaque tête travaille dans un petit sous-espace (128) ;
* elle voit tous les tokens.
- Têtes indépendantes → 1 grosse matmul batchée. Concat + Linear (W_O, apprise) pour recombiner.
- Coût en calcul ≈ celui d'une seule grosse tête (même dimension totale).
- En génération, on garde K et V de tous les tokens passés (KV cache) → beaucoup de mémoire, en particulier pour MHA
- GQA (2023) : plusieurs têtes Q partagent les mêmes K, V → cache ÷8 chez Llama 3 70B, quasiment sans perte.
- MQA : 1 seule tête KV → plus de perte.

→ *Transition* : après l'attention (communication), le 2e bloc : le MLP (réflexion)… qui contient la majorité des paramètres.

The MLP block (Feed Forward)



Intuition: MLP as Associative Factual Memory

- **Token-independent :** Each vector is processed completely alone (no inter-token mixing).
- **W_1 asks thousands of 'questions' :** Parallel pattern detectors (e.g., *'Is this token Michael Jordan?'*).
- **Non-linearity acts as a filter :** Retains only activations with positive matches ('yes') and silences the rest.
- **W_2 writes the associated 'answers' :** Adds factual info back to the vector (+ *'plays basketball'*, *'Chicago Bulls'*).

Parameter Budget: MLP accounts for ~2/3 of parameters in GPT-3, ~80% in Llama 3.3-70B

- Après l'attention, chaque vecteur passe seul dans le même petit réseau à 2 couches. Ici, aucun échange entre tokens.
- Forme : agrandir (×4) → non-linéarité → réduire.
- Intuition 3b1b (ch. 7) :
* chaque ligne de W_1 pose une question au vecteur ;
* la non-linéarité ne laisse passer que les « oui » ;
* W_2 ajoute l'info associée → une partie des « faits » est stockée ici.
- Version moderne, SwiGLU :
* une branche « porte » (gate) module l'autre ;
* 3 matrices au lieu de 2 → on réduit d_{ff} pour garder à peu près le même nombre de params ;
* un peu meilleur à coût égal.
- Autour du bloc : normalisation (LayerNorm, aujourd'hui RMSNorm) + résiduel (chaque bloc *ajoute* sa correction).
- (option) MoE (Mixtral, DeepSeek-V3) : plusieurs MLP « experts », chaque token n'en utilise que quelques-uns → beaucoup de params, peu de calcul par token.

→ *Transition* : on a toutes les briques. On compte les paramètres d'un vrai LLM.

Where are the parameters? — Llama 3.3-70B



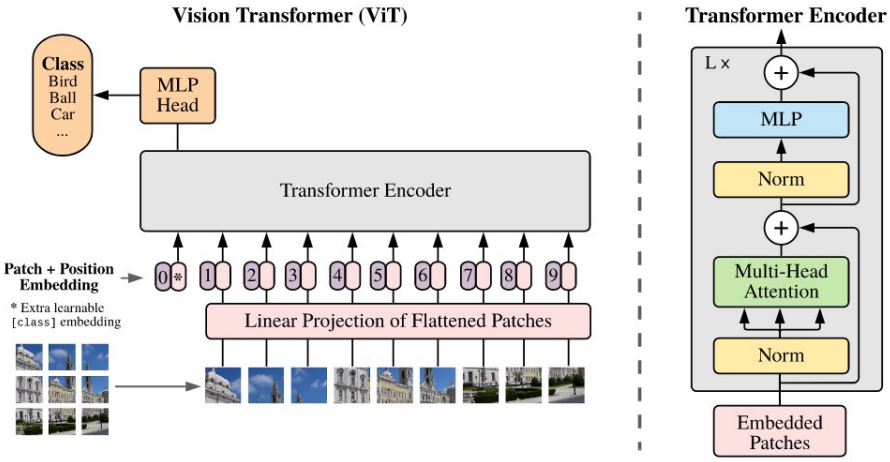
- Méthode : lire le `config.json` (Hugging Face) et recalculer à la main. Bon réflexe pour estimer la mémoire avant un job.
- Embeddings : 1 vecteur de 8192 par token du vocabulaire (128k) → 1 B. Autant en sortie.
- Attention : Q et O pleins ; K et V 8× plus petits grâce à GQA (8 têtes KV × 128).
- MLP : 3 grosses matrices → 80 % du modèle. Les normes sont négligeables (~1 M).
- Mémoire :

* 141 Go en bf16 → ne tient pas sur 1 GPU, même en inférence ;
* entraînement : 16 o/param (RAPPEL !) = poids bf16 (2) + gradients (2) + copie fp32 (4) + Adam (4 + 4)
* → ~1,1 To → des dizaines de GPU → parallélisme de modèle.

- (option) Calcul : ≈ 2 FLOP/param/token en inférence, ≈ 6 en entraînement → ~140 GFLOP par token généré.

→ *Transition* : et pour les images ? Même architecture, avec une seule question : comment faire des tokens à partir d'une image ?

Vision Transformer (ViT, 2020)



- Patches are the 'words' of the image (« An Image is Worth 16x16 Words »)
- + learned position
- Token [CLS] (like BERT): its output is used for classification
- Encoder only: no mask; each patch sees the entire image

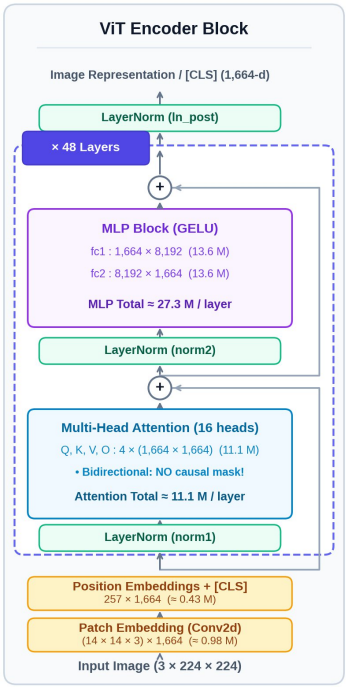
Resolution ×2 → tokens ×4 → attention ×16

[12]

- Google, 2020. Idée : ne rien changer au Transformer, seulement l'entrée.
- découpe en Patches, ensuite aplati → projection linéaire → embedding + position.
- [CLS] : un token appris en plus, qui récolte l'info de toute l'image → tête de classification.
- Encodeur seul (famille BERT) : pas de masque.
- Peu d'a priori « image » (pas de convolution/extraction de feature) → il faut beaucoup de données.
- Le ViT ne bat les CNN qu'après pré-entraînement sur ImageNet-21k ou JFT-300M.
- HPC : nb de tokens $\propto$ résolution², attention $\propto$ tokens² → la haute résolution coûte très cher.

→ *Transition* : comptons les paramètres d'un gros ViT, le ViT-G/14.

Where are the parameters? — ViT-G/14 (1.84 B)



OpenCLIP config.json — ViT-bigG/14 (same dimensions as ViT-G/14)

```
{
  "hidden_size":          1664,   Hidden state dimension (d)
  "intermediate_size":    8192,   MLP dimension (4.92 × d)
  "num_hidden_layers":    48,     Total Transformer encoder layers
  "patch_size":           14,     Patch resolution (14 × 14 px)
  "num_attention_heads":  16,     Q/K/V heads (head_dim = 104)
  "num_channels":         3,      Input image channels (RGB)
  "image_size":           224     Input image resolution (16×16)
}
```

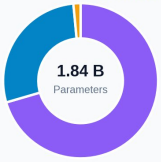
config: CLIP ViT-bigG/14 (OpenCLIP), identical dimensions to ViT-G/14 (Zhai et al., 2021)

Parameter Accounting (Vision Transformer)

1. **Input Embeddings (Patch + Positions) : ≈ 1.41 M (< 0.1%)**
   - Patch embed : (14 × 14 × 3) × 1,664 = 588 × 1,664 ≈ 0.98 M (3 RGB)
   - Position embed : 257 × 1,664 = (16×16 patches + [CLS]) ≈ 0.43 M
2. **Attention (per layer) — Full MHA (no GQA) : ≈ 11.08 M / layer**
   - Q, K, V projections : 3 × (1,664 × 1,664) = 3 × 2.77 M = 8.31 M
   - Output projection : 1 × (1,664 × 1,664) = 1 × 2.77 M = 2.77 M
3. **Classical MLP (per layer) — 2 matrices + GELU : ≈ 27.26 M / layer**
   - fc1 (expand) : 1 × (1,664 × 8,192) = 1 × 13.63 M = 13.63 M
   - fc2 (project) : 1 × (8,192 × 1,664) = 1 × 13.63 M = 13.63 M
4. **Across 48 Layers (× 48) : ≈ 1.84 B / model**
   - Attention (48 layers) : 48 × 11.08 M ≈ 0.53 B (28.9% of model)
   - MLP blocks (48 layers) : 48 × 27.26 M ≈ 1.31 B (71.1% of model)

Total Parameters : ≈ 1.84 B ✓
1.41 M (embeddings) + 0.53 B (attention) + 1.31 B (MLP) = 1,841.7 M

Parameter Breakdown



- **MLP : 71%** — 1.31 B (2 mat. / layer)
- **Attention : 29%** — 0.53 B (full MHA, 4 mat.)
- **Input : < 0.1%** — 1.4 M (patch + pos)

vs. Llama 70B (Vision vs. Language)

- **No vocabulary table :** In & out nearly free (< 0.1% vs 2.1 B in Llama)
- **38× fewer parameters (1.84 B vs. 70.6 B)**
- **Token explosion with image resolution :** At 518²: 37 × 37 = 1,369 tokens / image → heavy O(N²)

HPC Memory Footprint — Jean Zay / IDRIS

Inference (BF16 weights) :
- 1.84 B × 2 bytes ≈ 3.7 GB → easily fits on 1 GPU!

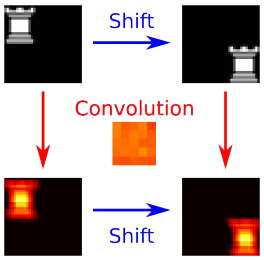
Training (Adam + Mixed Precision) :
- Optimizer states: ~30 GB (~16 B / param)
- **High token count (1,369 at 518² resolution)**
  → **Activations quickly saturate memory!**
- ViT-G training: > 30,000 TPUv3 core-days (Zhai et al., 2021)

- Même recette : config → calcul.
- Q, K, V, O pleins : pas de GQA, car pas de génération donc pas de KV cache.
- MLP classique (2 matrices, GELU) → 71 % des params.
- Différence avec un LLM : pas de grosse table d'embedding. Le coût vient du nombre de tokens.
- Mémoire : les poids tiennent sur 1 GPU, l'entraînement non (états + activations).
- ViT-G (2021) : 90,45 % sur ImageNet, pour > 30 000 jours-cœurs TPUv3 (cf. tableau CoAtNet).

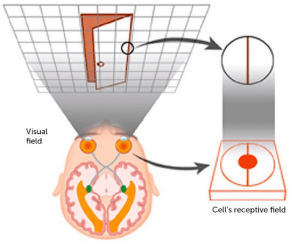
→ *Transition* : le ViT a peu de « bon sens » visuel, il lui faut des données énormes. CoAtNet remet des convolutions.

CoAtNet (2021): Convolution + Attention

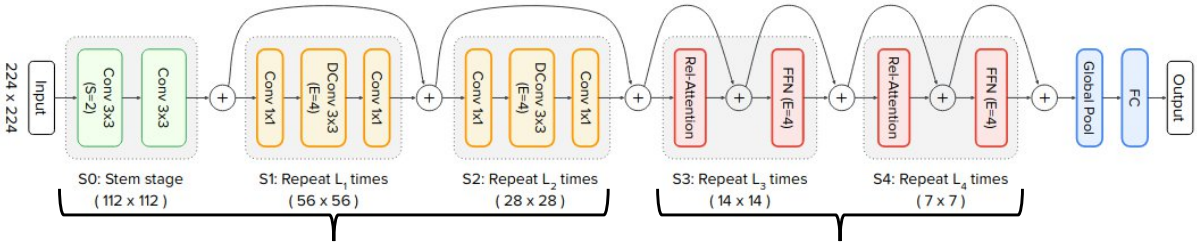
Convolution: looks locally, same filter everywhere → learns fast, with little data



Attention: looks at whole image → more expressive, but data-hungry



Input = raw image, no patches. Each "pixel" of the 14×14 map becomes a token



Convolutions (MBConv): high resolution, local features

Transformer : reduced map (14×14, then 7×7) → few tokens, global context

Replaces a CNN (ResNet) in a code without changing anything

- Google, 2021 : le meilleur des deux mondes.
- Début en CNN (MBConv, comme EfficientNet) :
* travaille à haute résolution, sur des motifs locaux ;
* bon a priori : équivariance en translation (chat en haut ou en bas → même réponse du filtre) → apprend avec moins de données.
- Fin en Transformer, sur la carte réduite → peu de tokens → attention globale abordable (n^2 petit).
- Pas de découp patchs en entrée → s'utilise comme un ResNet
- « Rel-Attention » : l'attention tient aussi compte de la position relative (clin d'œil à la convolution).

→ *Transition* : quelles tailles, pour quels résultats ?

CoAtNet: Sizes and Results (2021)

	Models	Eval Size	#Params	#FLOPs	TPUv3-core-days	Top-1 Accuracy
JFT-300M pre-training	ResNet + ViT-L/16	384^2	330M	-	-	87.12
	ViT-L/16	512^2	307M	364B	0.68K	87.76
	ViT-H/14	518^2	632M	1021B	2.5K	88.55
	NFNet-F4+	512^2	527M	367B	1.86K	89.2
	CoAtNet-3†	384^2	168M	114B	0.58K	88.52
	CoAtNet-3†	512^2	168M	214B	0.58K	88.81
	CoAtNet-4	512^2	275M	361B	0.95K	89.11
	CoAtNet-5	512^2	688M	812B	1.82K	89.77
JFT-3B	ViT-G/14	518^2	1.84B	5160B	>30K◇	90.45
	CoAtNet-6	512^2	1.47B	1521B	6.6K	90.45
	CoAtNet-7	512^2	2.44B	2586B	20.1K	**90.88**

JFT = Google's internal dataset (300 M then 3 B images)
evaluation on ImageNet-1k (1.28 M images)
ImageNet-21k: ~14 M images

same accuracy (90.45%), 3.4× fewer FLOPs,
> 4.5× fewer training computations

- 8 tailles : de CoAtNet-0 (25 M) à CoAtNet-7 (2,4 B).
- Ici : pré-entraînement sur JFT, puis fine-tuning et test sur ImageNet-1k.
- Lecture clé : CoAtNet-6 égale ViT-G/14, avec ~3× moins de FLOPs et ~5× moins de calcul d'entraînement.
- CoAtNet-7 : 90,88 % top-1, n° 1 en 2021.
- 2 messages :

* un bon a priori (les convolutions) = de l'efficacité ;
* les gros modèles ont besoin d'énormément de données (1,28 M images ne suffisent pas).

→ *Transition* : on résume ce qu'il faut retenir.

Key Takeaways: Transformers



- Entire sequence in parallel (≠ RNN), and each token sees all the others (≠ CNN, local)
- Attention = communication between tokens (cost in n^2); MLP = processing per token (majority of the weights)
- Scale very well: from 65 million (2017) to hundreds of billions of parameters
- Few prior assumptions → require very large datasets
- **No longer fit on a single GPU: parallelism (training), optimizations (inference)**

Quelques messages clés sur les transformers :

- 1) Calcul parallèle → GPU ; vision globale (le CNN, lui, est local).
- 2) Deux blocs : l'attention (coût en n^2) et le MLP (les params).
- 3) Plus gros + plus de données = meilleur (lois d'échelle) → on les fait énormes
- 4) Peu d'a priori → beaucoup de données (ViT vs CoAtNet).

→ *Transition* : 2 problèmes HPC : les entraîner (parallélisme) et les servir (optimiser l'inférence, la suite).

Optimizing Inference for Very Large Models

Less memory, more speed… without (too much) loss in quality

Distillation ◂

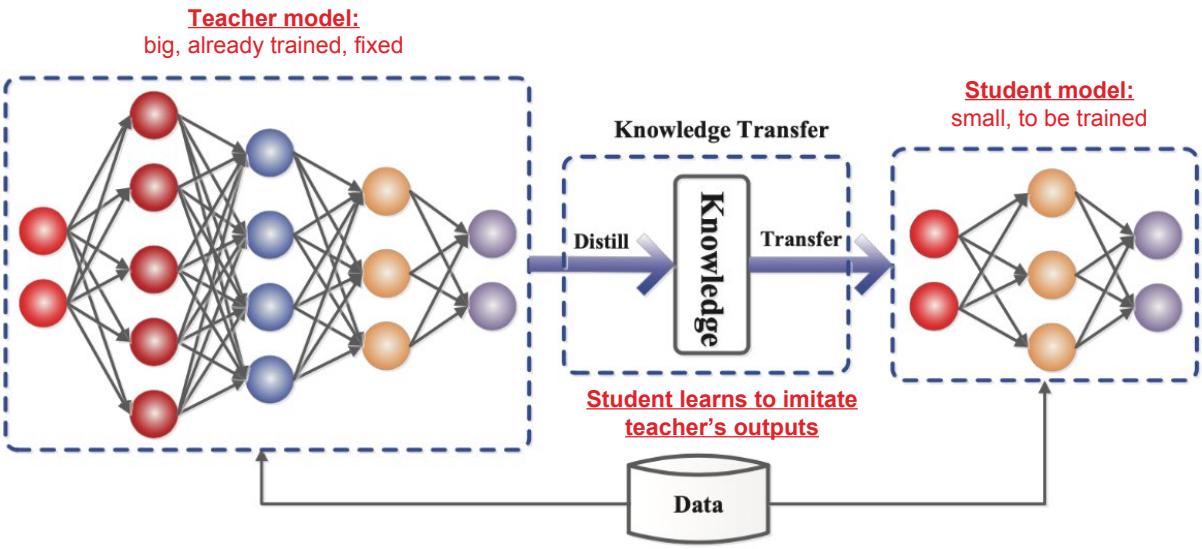
Quantization ◂

Pruning ◂

KV cache & PagedAttention ◂

- Entraîné une fois, utilisé des millions de fois → sur la durée, c'est l'inférence qui coûte.
- 3 leviers sur le modèle :
* le rendre plus petit (distillation) ;
* utiliser moins de bits (quantification) ;
* garder moins de poids (pruning).
- 1 levier sur la génération : KV cache + PagedAttention
→ *Transition* : premier levier, c'est le concept d'un gros prof et un petit élève.

Distillation: from big teacher to small student



Examples:

DistilBERT (2019) • DINOv3 (2025) : ViT-7B → ViT-B/L, ConvNeXt •
Llama 3.2 1B/3B (2024) • DeepSeek-R1-Distill (2025)

- Les gros modèles sont meilleurs, mais chers à servir.
- Idée (Hinton, 2015) : entraîner un petit modèle à imiter le gros.
- Pourquoi c'est mieux qu'un entraînement seul du petit ? Le prof donne plus qu'un label, il donne toute sa distribution.

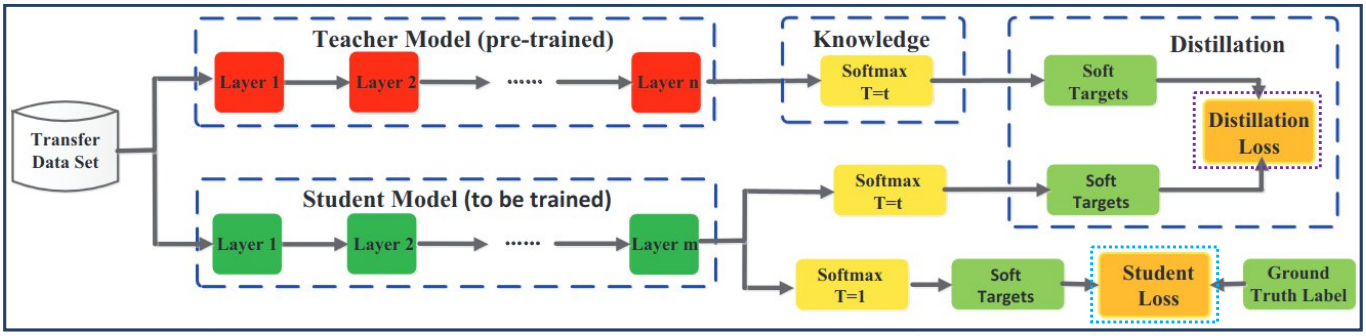
* Ex : « 70 % chat, 25 % chien, 0,1 % voiture » → un chat ressemble plus à un chien qu'à une voiture.

- On garde l'essentiel de la qualité pour une fraction du coût.
- C'est partout :

* vision : DINOv3 (Meta, 2025) distille son ViT-7B en ViT-B/L et ConvNeXt ;
* LLM : Llama 3.2 1B/3B (logits de Llama 3.1 8B/70B), DeepSeek-R1-Distill.

→ *Transition* : concrètement, quelle loss ?

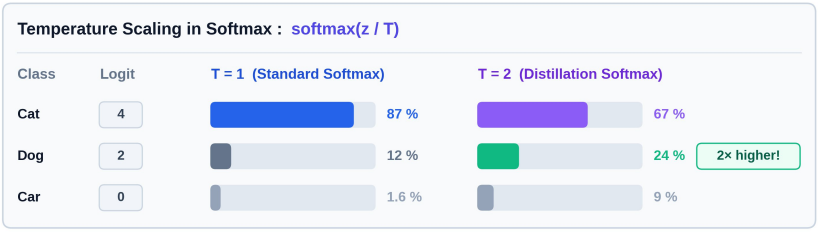
Distillation: loss function



$$\mathcal{L} = \mathcal{L}_{\text{distil}}\left(\text{softmax}\left(z_{\text{teacher}}/T\right),\ \text{softmax}\left(z_{\text{student}}/T\right)\right) + \lambda\,\mathcal{L}_{\text{CE}}\left(y_{\text{true}},\ y_{\text{student}}\right)$$

KL (most common), cross-entropy, MSE sur les logits

Ex.: DistilBERT,
40 % less params, 60 % faster,
97 % of the performances of BERT



- 2 termes :

* imiter le prof : comparer les 2 distributions (souvent avec la KL) ;

* si on a les labels : la loss classique, pondérée par λ.

- La température (celle qui règle la « créativité » en génération) :

* T > 1 aplatit la distribution → fait ressortir les « 2e choix » du prof ;

* même T > 1 pour les deux pendant l'entraînement, puis T = 1.

- Variante : imiter aussi les états internes (DistilBERT ajoute une similarité cosinus).

- LLM : distillation sur les logits (Llama 3.2), ou simple fine-tuning sur des textes générés par le prof (DeepSeek-R1-Distill).

- Limite : il faut faire tourner le prof sur tout le dataset → coûteux, mais une seule fois.

→ *Transition* : deuxième levier, garder le même modèle mais stocker chaque poids sur moins de bits.

Quantization: why?

Format	Bytes	A100	H100 SXM
FP32	4	19.5 TFLOPS	67 TFLOPS
TF32 (Tensor Core)	4	156	495
BF16 / FP16	2	312	989
FP8	1	—	1,979
INT8	1	624 TOPS	1,979 TOPS
Memory bandwidth		2.0 TB/s	3.35 TB/s

- Bits ÷2 → memory ÷2 and Tensor Core throughput ×2
- With 2:4 pruning, another ×2 (see pruning later); Blackwell: FP4

HPC for LLMs:
In the generation stage, all weights are re-read for each token
→ bandwidth is the limiting factor.
70 B in BF16 = 141 GB read per token; in 4-bit ≈ 35 GB
→ up to ~4× faster

- 2 gains :
* mémoire : moins d'octets par poids ;
* calcul : Tensor Cores 2× plus rapides à chaque fois qu'on divise les bits par 2.
- A100 : INT8 = 2× BF16. H100 ajoute le FP8, Blackwell le FP4.
- x 2 aussi avec pruning 2:4 (on y reviendra)
- Le gain clé pour les LLM :
* en génération, on relit tous les poids à chaque token → limité par la mémoire, pas par le calcul ;
* des poids 4× plus petits ≈ jusqu'à 4× plus vite.
- Exemple 2025 : gpt-oss-120b a ses poids MoE en 4 bits (MXFP4) → il tient sur 1 GPU de 80 Go.
→ *Transition* : comment passe-t-on de flottants à des entiers 8 bits ?

Quantizing weights: how?



$$s = \frac{\max|w|}{127}, \qquad q = \text{round}\left(\frac{w}{s}\right), \qquad \hat{w} = s \cdot q$$

Example : $s = 0.01$; $w = 0.4237 \rightarrow q = 42 \rightarrow \hat{w} = 0.42$

⚠ Outliers Issue

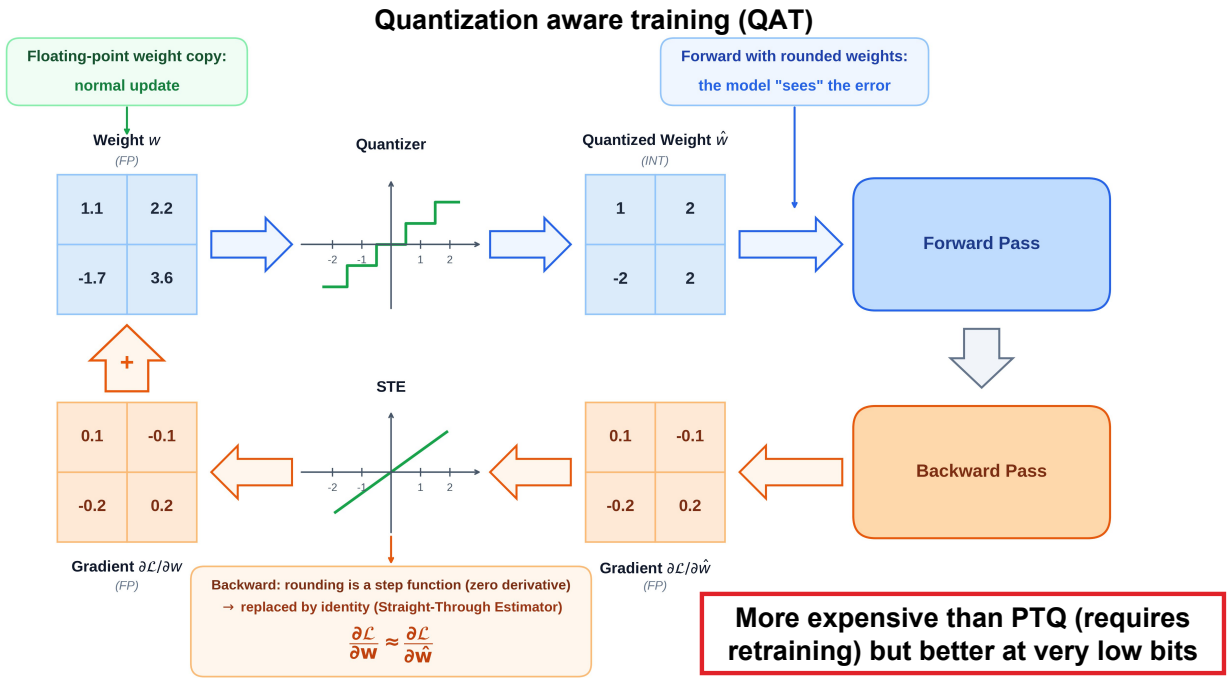
- **A single weight at** $12.7 \rightarrow s = 0.1$
  → all small weights collapse to 0 **or** ± 1 **!**
- **Solution: One scale per row or per group**
  (e.g., separate scale for every 128 weights)

- **PTQ (Post-Training Quantization):**
  From a few minutes to a few hours,
  with a little calibration data
  (GPTQ, AWQ, GGUF, FP8...)
- **Precision Trade-offs:**
  8 bits ≈ lossless
  4 bits: small loss, very widely used
  < 4 bits: degrades rapidly
- **Quantization Scope:**
  Weights only (W4A16) → memory savings
  Weights + activations (W8A8, FP8) → compute speedup too

⚠ Always re-evaluate the quantized model!
Performance drops can occur depending on tasks.

- Un « cast » direct est impossible : les poids valent ~0,01
→ presque tout serait arrondi à 0. Il faut une échelle
- Principe : étaler la plage des poids sur les 255 entiers. On stocke q (8 bits) + s (1 flottant par tenseur,).
- Pour calculer :
* soit on déquantifie à la volée (s \dot q) ;
* soit on calcule directement en entiers, si les activations sont aussi quantifiées.
- L'ennemi, ce sont les valeurs extrêmes :
* elles étirent l'échelle → perte de précision → échelles par petits groupes ou par ligne ;
* dans les LLM, certaines activations ont de gros outliers → méthodes dédiées (LLM.int8, SmoothQuant…).
- La PTQ est le cas le plus courant : modèle déjà entraîné + calibration (GPTQ, AWQ, GGUF de llama.cpp, FP8 sur H100).
- Si ça perd trop → ré-entraînement court avec quantification simulée (slide suivante)
→ *Transition* : comment entraîner avec des poids arrondis, alors que l'arrondi n'est pas dérivable ?

Quantization during training



- Problème : l'arrondi est un escalier → dérivée nulle presque partout → aucun gradient.
- L'astuce :
* garder une copie flottante des poids ;
* faire le forward avec la version arrondie → le modèle apprend à vivre avec l'erreur ;
* au backward, faire comme si l'arrondi était l'identité (STE, Bengio 2013) → le gradient passe tel quel ;
* mettre à jour la copie flottante.
- À la fin, on garde la version quantifiée.
- Coût : un ré-entraînement (court) → réservé aux bas bits (4 et moins) ou au cas où la PTQ perd trop.
- La même astuce STE sert pour le pruning (2 slides plus loin).
- (option) L'entraînement lui-même passe en basse précision : DeepSeek-V3 a été entraîné en FP8 (2024)

→ *Transition* : troisième levier, supprimer des poids. Point de départ : une loterie.

Pruning: the lottery ticket

The Lottery Ticket Hypothesis.
A randomly-initialized, dense neural network contains a subnetwork that is initialized such that—when trained in isolation—it can match the test accuracy of the original network after training for at most the same number of iterations.

Frankle & Carbin, 2019

[24, 25, 26]

- Constat : beaucoup de poids d'un réseau entraîné servent à peu de chose.

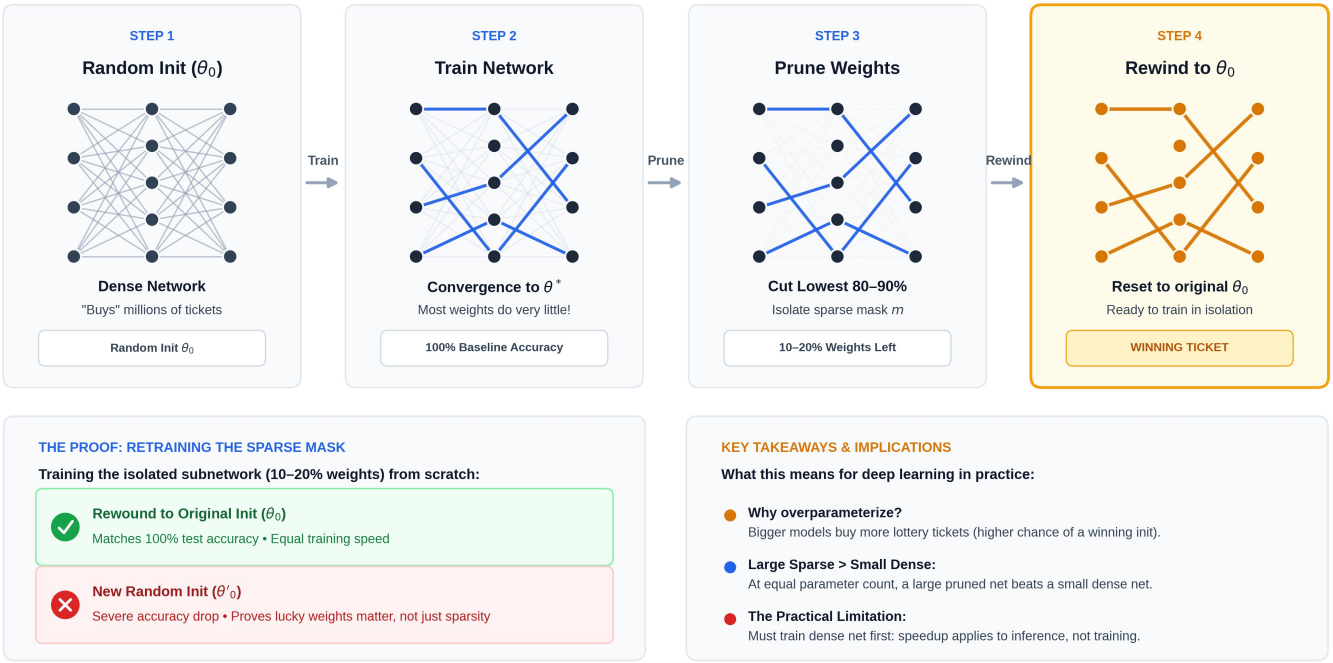
- Point de départ: l'hypothèse du ticket gagnant: « Un gros réseau aléatoire contient un petit sous-réseau qui, entraîné seul depuis la même initialisation, atteint la même précision »

- Autrement dit, l'idée du ticket gagnant c'est que:

* l'initialisation aléatoire est une loterie, et un gros réseau contient beaucoup de « sous-réseaux » = tickets ;

* parmi eux, un sous-réseau « gagnant » existe dès le départ, aussi performant que le réseau complet

Pruning: the lottery ticket



- Pour le trouver : entraîner, couper les plus petits poids, revenir à l'init, recommencer → 10–20 % des poids suffisent.
- Important de reprendre les mêmes poids initiaux !
- Messages à retenir:

* Intuition bonus : c'est une des explications au fait que les gros réseaux s'entraînent mieux (plus de tickets).
* à nombre de poids non nuls égal, un gros réseau creux bat un petit réseau dense.
* Limite : il faut d'abord entraîner le gros réseau → le gain est à l'inférence, pas à l'entraînement.

→ *Transition* : en pratique, quels poids couper, et est-ce vraiment plus rapide ?

Pruning: why?



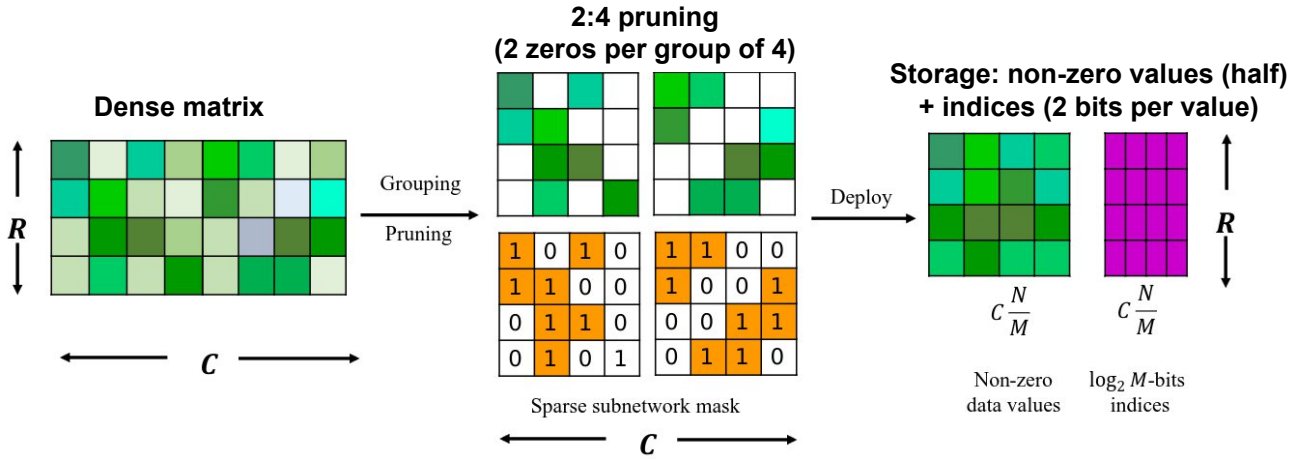
- Critère : couper les plus petits |w|. C'est simple, et souvent aussi bon que des méthodes compliquées [25].
- LLM : SparseGPT, Wanda → ~50 % de zéros en une passe, sans ré-entraînement.
- Mais des zéros éparpillés n'accélèrent pas le GPU : pour lui, la matrice reste « dense ». Il faut une structure :

* structuré : enlever des neurones, des têtes ou des couches → un vrai modèle plus petit (ex : Llama 3.2 1B/3B = pruning de Llama 3.1 8B + distillation) ;
* semi-structuré 2:4 → accéléré par les Tensor Cores.

- Après le pruning : fine-tuning pour récupérer (masque + STE [20]).

→ *Transition* : zoom sur le 2:4, le format accéléré par les GPU NVIDIA.

2:4 Pruning on NVIDIA GPUs



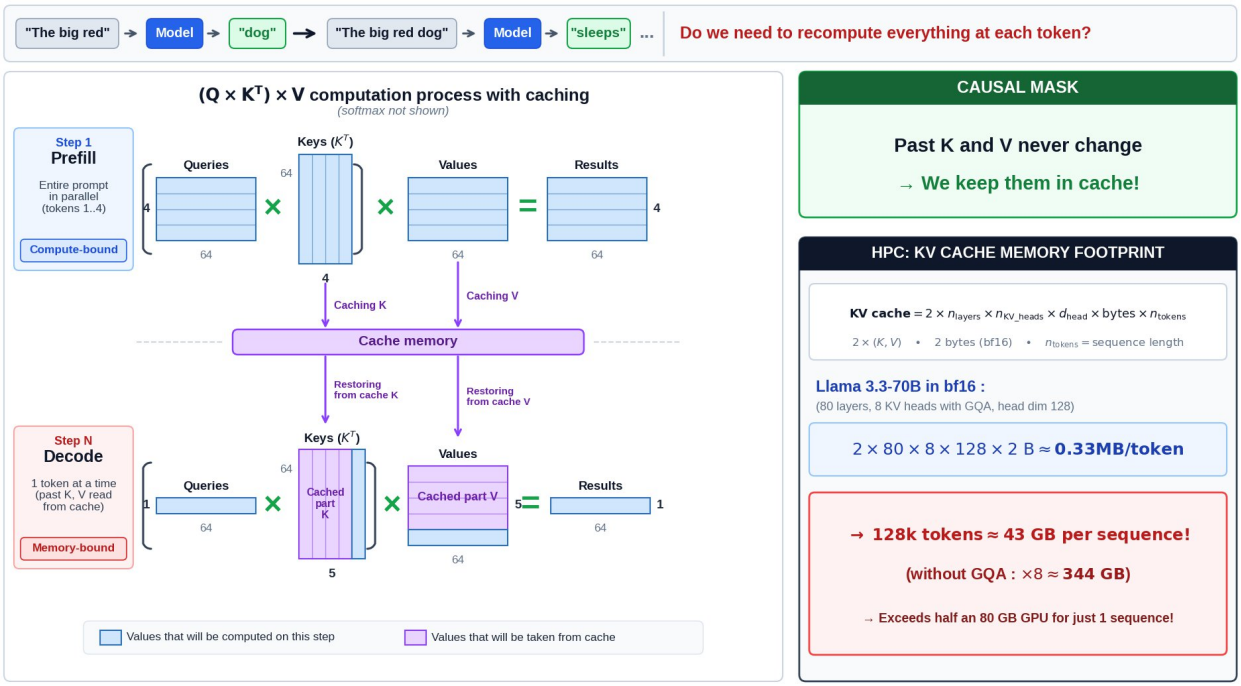
- A100/H100 Tensor Cores skip zeros → up to ×2 on matmuls
- Weights ~÷2 in memory ; combinable with INT8 / FP8
- Actual end-to-end gain < ×2, and a loss of precision that needs to be recovered (fine-tuning)”

- 2:4 : dans chaque groupe de 4 poids consécutifs, au moins 2 zéros → 50 % de zéros, mais réguliers.
- Stockage compressé : les 2 valeurs + leur position (2 bits).
- Les Tensor Cores ne calculent que les non-nuls → débit théorique ×2
- En réalité :

* gain de bout en bout < ×2, car le reste du modèle ne change pas ;
* imposer 50 % de zéros fait perdre de la précision, qu'il faut récupérer ;
* c'est moins utilisé que la quantification.

→ *Transition* : on a allégé le modèle. Pour les LLM, il reste un gros poste d’opti possible: la génération token par token.

KV Cache



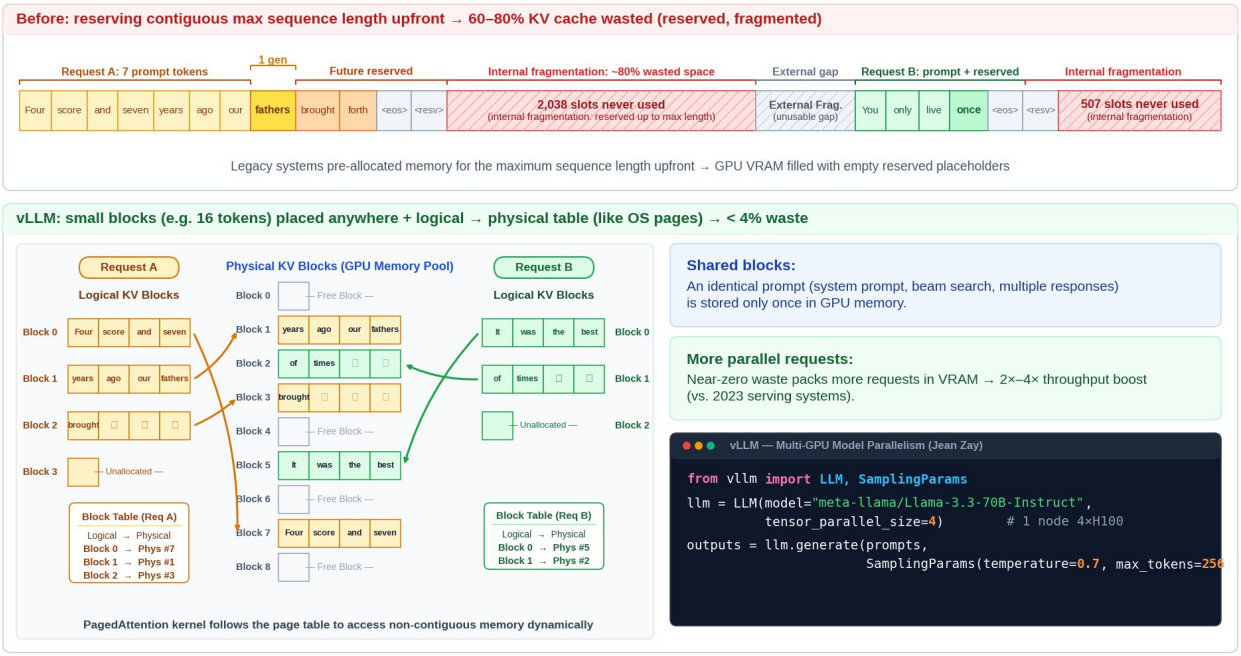
- La génération est une boucle : 1 token, on l'ajoute, on relance → un passage complet du modèle par token.
- Sans cache, on recalcule à chaque token les K et V de tout le texte → gaspillage énorme.
- Masque causal : les tokens passés ne voient pas le futur → leurs K, V ne changent pas → on les stocke.
- Pour un nouveau token, on calcule seulement ses vec Q, K, V ; son vec Q est comparé à toutes les K du cache.
- 2 phases :

* prefill : le prompt d'un coup, limité par le calcul ;
* decode : token par token, limité par la mémoire (on relit les poids + le cache).

- Le prix, c'est la mémoire : pour Llama 70B, 43 Go pour 128k tokens → plus d'un demi-GPU pour *une seule* séquence.
- D'où GQA (÷8), le cache en FP8, etc.
- Autre problème : la longueur de la réponse est inconnue → combien de mémoire réserver ?

→ *Transition* : vLLM répond avec une idée des années 60, la mémoire virtuelle.

Paged Attention



Kwon et al., SOSP 2023 — now ubiquitous (vLLM, SGLang, TensorRT-LLM, TGI)

- Problème : on ne connaît pas la longueur de la réponse. Les systèmes classiques réservaient d'un bloc la longueur max pour chaque requête :

* mémoire réservée mais vide, et des trous entre requêtes ;

* mesuré : 60–80 % du KV cache perdu.

- Moins de mémoire utile → moins de requêtes en parallèle → moins de débit.

- L'idée de vLLM (2023), c'est la mémoire virtuelle d'un OS :

* le cache est découpé en blocs (ex : 16 tokens) ;

* les blocs sont alloués à la demande, n'importe où ;

* chaque requête a une table : bloc logique → bloc physique.

- Les kernels d'attention classiques veulent K, V contigus → vLLM a son propre kernel, qui suit la table. D'où le nom

- Bonus : des blocs partagés entre requêtes, qu'on ne copie que quand elles divergent.

- Résultat : moins de 4 % de gaspillage → batch bien plus gros → débit ×2–4.

- Sur Jean Zay, vLLM tient en 3 lignes ; `tensor_parallel_size=4` découpe le modèle sur 4 GPU → c'est déjà du parallélisme de modèle.

→ *Transition* : récap (S27bis), puis le parallélisme.

Recap : which technique for which problem

Technique	What it reduces	Typical gain	Cost
Distillation	number of parameters	model several times smaller	train the student (+ have the teacher run it)
Quantization	bits per weight (16 → 8 → 4)	memory ÷2 to ÷4, faster generation	loss of precision (at 4 bits and below)
Pruning	non-zero weights	2:4 → up to ×2 on matmuls	limited real-world gain, fine-tuning
GQA	KV cache size	÷8 (Llama 3 70B)	architecture choice, before training
KV cache	calculations redone for each token	much faster generation	memory (0.33 MB/token for Llama 70B)
PagedAttention	wasted KV cache	throughput ×2 - 4	none (just use vLLM)

Cumulative: e.g., distilled model + 4 bits + vLLM

- 3 techniques sur le modèle, 3 sur la génération. Elles se cumulent.
- Ordre pratique :
1. vLLM (gratuit) ;
2. quantification (8 bits ≈ sans perte, 4 bits souvent OK) ;
3. distillation ou pruning pour aller plus loin (coûteux).
- Toujours mesurer : la qualité (benchmark de *sa* tâche) et la vitesse réelle.

→ *Transition* : si le modèle ne tient toujours pas sur 1 GPU, ou s'il faut l'entraîner → parallélisme de modèle